

ITC MPSI, DS n°1

Autour des nombres premiers

I) Questions de cours

Répondez succinctement aux questions suivantes :

1. Comment affiche-t-on une variable x en Python ?
2. Quelle est la différence entre une boucle `for` et une boucle `while` ? Quand utilise-t-on l'une ou l'autre ?
3. Comment définit-on un commentaire en Python ?
4. Quelle notation permet d'indiquer qu'un programme a une complexité *linéaire* en fonction de n ?
5. Si un programme fait $\frac{n^3+5n^2+8}{n+10}$ opérations, quelle est sa complexité en fonction de n ?
6. Citez trois algorithmes de tri. L'un d'eux commence par un i . Détaillez les opérations effectuées par cet algorithme sur la liste $[3, 2, 5, 1]$ lorsqu'on trie en ordre croissant.

II) Préliminaires

Ce sujet est inspiré du sujet d'informatiques de 2019 aux Mines.

Préambule

Chiffrer les données est nécessaire pour assurer la confidentialité lors d'échanges d'informations sensibles. Dans ce domaine, les nombres premiers servent de base au principe de clés publique et privée qui permettent, au travers d'algorithmes, d'échanger des messages chiffrés. La sécurité de cette méthode de chiffrement repose sur l'existence d'opérations mathématiques peu coûteuses en temps d'exécution mais dont l'inversion (c'est-à-dire la détermination des opérandes de départ à partir du résultat) prend un temps exorbitant. On appelle ces opérations « fonctions à sens unique ». Une telle opération est, par exemple, la multiplication de grands nombres premiers. Il est aisé de calculer leur produit. Par contre, connaissant uniquement ce produit, il est très difficile de déduire les deux facteurs premiers. Le sujet étudie différentes questions sur les nombres premiers.

Il n'est pas nécessaire d'avoir réussi à écrire le code d'une fonction pour pouvoir s'en servir dans une autre question.

Définitions, rappels et notations

- Un nombre premier est un entier naturel qui admet exactement deux diviseurs : 1 et lui-même. Ainsi 1 n'est pas considéré comme premier.
- Quand une fonction Python est définie comme prenant un « nombre » en paramètre cela signifie que ce paramètre pourra être indifféremment un flottant ou un entier.
- On note $\lfloor x \rfloor$ la partie entière de x .
- la notation $1e-3$ en python représente 10^{-3} .
- `abs(x)` renvoie la valeur absolue de x . La valeur renvoyée est du même type de données que celle en argument.
- `int(x)` convertit vers un entier. Lorsque x est un flottant positif ou nul, elle renvoie la partie entière de x , c'est-à-dire l'entier n tel que $n \leq x < n + 1$.
- `round(x)` renvoie la valeur de l'entier le plus proche de x . Si deux entiers sont équidistants, l'arrondi se fait vers la valeur paire.
- `floor(x)` / `ceil(x)` renvoient la valeur du plus grand / petit entier inférieur ou égal à x .
- `log(x)` renvoie sous forme de flottant la valeur du logarithme népérien de x .
- Pour importer une fonction d'un module, on écrit `from module import fonction1, fonction2, fonction3` (en remplaçant évidemment par les noms appropriés)

Consignes d'écriture du code

- Les noms de variables doivent être explicites, et le code doit être commenté lorsque c'est nécessaire. C'est d'autant plus nécessaire si vous définissez des fonctions auxiliaires.
- L'indentation de votre code doit être parfaitement claire. À cet effet, vous devez signaler les niveaux d'indentation par des lignes verticales.

Q1 – Dans un programme Python on souhaite pouvoir faire appel aux fonctions `log`, `sqrt`, `floor` et `ceil` du module `math` (`round` est disponible par défaut). Écrire des instructions permettant d'avoir accès à ces fonctions et d'afficher le logarithme népérien de 0.5.

Q2 – Écrire une fonction `sont_proches(x, y)` qui renvoie `True` si la condition suivante est remplie et `False` sinon

$$|x - y| \leq \text{atol} + |y| \times \text{rtol} \quad (1)$$

où `atol` et `rtol` sont deux constantes, à définir dans le corps de la fonction, valant respectivement 10^{-5} et 10^{-8} . Les paramètres `x` et `y` sont des nombres quelconques.

Q3 – On donne la fonction `mystere` ci-dessous. Que renvoie `mystere(1001, 10)` ? Le paramètre `x` est un nombre strictement positif et `b` un entier naturel non nul.

```
def mystere(x, b):  
    if x < b:  
        return 0  
    else :  
        return 1 + mystere(x / b , b)
```

Q4 – Exprimer ce que renvoie `mystere` en fonction de la partie entière d'une fonction usuelle.

Q5 – On donne le code suivant :

```
pas = 1e-5  
x2 = 0  
N = 100000  
for i in range(N):  
    x1 = (i + 1) * pas  
    x2 = x2 + pas  
print("x1 : " , x1)  
print("x2 : " , x2)
```

L'exécution de ce code produit le résultat :

```
x1: 1.0  
x2: 0.9999999999980838
```

a) Commenter.

b) Donner la complexité de ce code en fonction de `N`.

III) Génération de nombres premiers

Le crible d'Ératosthène est un algorithme qui permet de déterminer la liste des nombres premiers appartenant à l'intervalle $\llbracket 1, n \rrbracket$. Son pseudo-code s'écrit comme suit :

Données : N un entier supérieur ou égal à 1.

Résultat : *liste_bool*, une liste de booléens.

```
1  liste_bool ← liste de N booléens initialisée à Vrai
2  Marquer comme Faux le premier élément de liste_bool
3  Pour  $i \leftarrow 2$  à  $\lfloor \sqrt{N} \rfloor$  faire
4      si  $i$  n'est pas marqué comme Faux dans liste_bool alors
5           $\quad$  Marquer comme Faux tous les multiples de  $i$  différents de  $i$  dans liste_bool.
6  retourner liste_bool
```

Figure 1: Crible d'Ératosthène

Q6 - Donnez une majoration de la complexité de l'algorithme du crible d'Ératosthène en fonction de N . (On demande un $O(\dots)$)

Q7 - Écrire la fonction `erato_iter( $n$ )` qui implémente l'algorithme de la Figure 1, pour un paramètre N qui est un entier supérieur ou égal à 1.

IV) Compter les nombres premiers

La question de la répartition des nombres premiers a été étudiée par de nombreux mathématiciens, dont Euclide, Riemann, Gauss et Legendre. On étudie dans cette partie les propriétés de la fonction $\pi(n)$, qui renvoie le nombre de nombres premiers appartenant à $\llbracket 1, n \rrbracket$.

Q8 - Écrire une fonction `pi( $N$ )` qui calcule la valeur exacte de $\pi(n)$ pour tout entier n de $\llbracket 1, n \rrbracket$. Les nombres premiers sont déduits de la liste *liste_bool* renvoyée par la fonction `erato_iter` de la question 7. On demande que `pi( $N$ )` renvoie son résultat sous la forme d'une liste de $[n, \pi(n)]$. Par exemple `pi(4)` renvoie la liste $[[1, 0], [2, 1], [3, 2], [4, 2]]$.

Il a été prouvé que $\frac{n}{\ln(n)-1} < \pi(n)$ pour tout $n > 5393$. On souhaite vérifier cette inégalité en se basant sur la fonction `pi( $N$ )` écrite en question 8.

Q9 - Écrire une fonction `verif_pi( $N$ )` qui renvoie `True` si l'inégalité est vérifiée jusqu'à N inclus, `False` sinon. Le paramètre N est un entier supposé supérieur ou égal à 5393.

V) Vérification

On suppose d'en cette partie qu'on nous donne une liste d'entiers, tous inférieurs à N , non triée.

Q10 - Écrire une fonction `est_liste_premier( $L, N$ )` qui vérifie que tous les entiers de la liste sont premiers. Elle doit renvoyer `True` si c'est le cas, `False` sinon. On demande d'utiliser la fonction `erato_iter` de la question 7.

Q11 - Écrire une fonction `trier_liste( $L$ )` qui trie *en place* la liste L donnée en argument, par ordre croissant. Vous pouvez utiliser l'algorithme de votre choix, mais on vous demande de nommer cet algorithme.

Q12 - Écrire une fonction `est_complete( $L, N$ )` qui vérifie si la liste L contient *exactement* tous les nombres premiers de $\llbracket 1, n \rrbracket$. Vous utiliserez pour cela les fonctions `trier_liste` et `erato_iter`.